Six Questions About Agentic AI Architecture

A detailed Day 2 handoff

Ricardo, Tomás, Simon, Luca, and Matheus
Foundation models / agentic AI architecture — 27 October 2026

What changed since the last version of this handoff

The talk is **40 minutes and 34 slides**, with speakers assigned in Day 1's order. Each topic is answered from one paper and stays self-contained; there is **no running example** this time. Real systems appear as evidence: **Pi** (a minimal coding agent), **Pi Durable** (Earendil's library for agents that survive crashes, released with Pi 1.0 on 1 October 2026) and **Hermes** (Nous Research's always-on agent). The right-hand column of each slide is read by the audience as key concepts; presenter directions are in the speaker notes. The deck, presenter guide and speaker notes are at `https://agentbox.tail289ef2.ts.net:8443/ml-ai-presentation/`; edit your own slide text in `slides/day2/` of the repository.

How to use this handoff. Everyone should read Section 1. Then read your own topic. Each topic section identifies the question it answers, the key points to cover, and a suggested transition sentence into the next topic. Nothing here depends on another speaker's slides existing yet.

The timings are a **40-minute** speaking plan, discussion included: Ricardo, Tomás, Simon, Luca and Matheus, in Day 1's order.

1 Before anyone writes a slide

1.1 The shared frame

Day 1 asked: how do you architect a system around a model that behaves statistically rather than deterministically? Day 2 asks the same question one layer up: how do you architect the *runtime* around a model that can now call tools, hold multi-step state, and act with some autonomy? Say this explicitly in the opening — it is the thread that ties Day 2 back to Day 1 without needing a shared example.

1.2 What each topic owes the audience

Every topic section below should open by naming its question in one sentence, give the audience enough vocabulary to follow the rest of the talk even if they skip the paper, and close with one concrete takeaway worth remembering after the slide is gone.

Avoid restating another topic's content — cross-references are given explicitly in each section's *Hands off to* note.

2 Topic 1: What is a harness, and why is it suddenly a named discipline?

Question to answer: what sits between a foundation model and a working system, and why does that layer now get its own name?

Key points. A coding or background agent is not just a model — it is a model plus a **harness**: the loop that calls the model, the tools it can invoke, how context is assembled and trimmed, and the extension surface that lets a team customize behavior without touching the core loop. "Harness engineering" was named as a discipline in early 2026. A 2026 source-code study audited eleven production harnesses (Claude Code, Codex CLI, Gemini CLI, Aider, OpenHands, Pi, Hermes, and others) across roughly four million lines of code, and defined **seven canonical subsystems** every harness implements in some form, plus a catalogue of **29 recurring design patterns**. Two findings are worth a slide on their own: none of the eleven systems import a general-purpose agentic framework for their core loop, and none retrieve code with vector embeddings — the field runs on hand-rolled loops and deterministic retrieval, not the heavier infrastructure many assume.

Brief illustration: naming a subsystem concretely

One subsystem is context management: deciding what from a long task history actually goes into the next model call. A minimal harness might simply summarize old history when a token budget is crossed; a more ambitious one might maintain layered memory files updated automatically. The point for this slide is the *category*, not any one system's specific implementation.

Hands off to Topic 2: once the audience has the seven-subsystem vocabulary, the next topic zooms into one subsystem in detail — the agent's own internal architecture — via a catalogue built specifically for foundation-model-based agents.

Reading anchor: Barbaste, Darrigol, Vu & Wiltberger (2026), arXiv:2609.00006.

3 Topic 2: How do you catalogue an agent's internal architecture?

Question to answer: if two teams are both building "an agent," what vocabulary lets them compare design choices precisely instead of talking past each other?

Key points. This paper performed a systematic literature review of foundation-model-based agents and distilled **18 architectural patterns**, each documented with its context, the forces that push toward or away from it, and its trade-offs — plus a decision model for choosing among them. Pick 2–3 patterns to go deep on rather than listing all 18; good candidates are ones with an intuitive hook, such as a planning pattern (how an agent breaks a goal into steps), a tool-use pattern (how it decides which tool to call and interprets the result), or a reflection pattern (how it checks its own output before acting further).

Hands off to Topic 3: having a vocabulary for architecture is only useful if you can also judge whether a given architecture is any good — which is exactly the gap the next paper addresses.

Reading anchor: Liu, Lo, Lu & Zhu (2025), Agent Design Pattern Catalogue*, Journal of Systems and Software, vol. 220.*

4 Topic 3: How do you evaluate an agent's architecture, not just its outputs?

Question to answer: traditional software architecture evaluation methods exist — why don't they transfer cleanly to agents?

Key points. Foundation-model-based agents have characteristics that break assumptions baked into older evaluation methods: **compound architecture** (many components, often other agents, calling each other), **autonomous and non-deterministic behavior** (the same input can lead to different action sequences), and **continuous evolution** (the underlying model or its prompt can change independently of the surrounding system). AgentArcEval is a method designed specifically for this, built around a **catalogue of agent-specific general scenarios** that a team adapts into concrete evaluation scenarios for their own system. The paper validates the method on a real-world tax copilot ("Luna"); mention this once as evidence the method works in practice, without reconstructing the case study in detail.

Hands off to Topic 4: one of AgentArcEval's own scenario categories concerns how well an agent's context and instructions are actually documented and maintained — which is precisely what the next topic studies empirically, in the wild.

Reading anchor: Lu, Zhao, Liu & Zhang (2026), AgentArcEval*, Journal of Systems and Software, vol. 232.*

5 Topic 4: What does context engineering actually look like in real projects?

Question to answer: vendors recommend maintaining a context file (like `AGENTS.md`) so an agent understands a project's conventions — do developers actually do this, and how?

Key points. This is an empirical study, not a proposal. Of **10,000** popular open-source repositories, only **466 (5%)** had an AI context file; the content analysis covers the **155 AGENTS.md files** among them and 453 commits of their history. Findings worth presenting as a short list rather than a wall of prose: there is **no established content structure yet** across projects; the way context is expressed varies along several styles — descriptive, prescriptive, prohibitive, explanatory, conditional — often mixed within one file; and a commit-level analysis gives early insight into how these files *evolve* over a project's lifetime, rather than being written once and left alone. Frame this as evidence that context engineering is still a genuinely open, unsettled practice, not a solved problem with one right answer.

Hands off to Topic 5: both the architecture-evaluation method and the context-engineering findings point toward the same open question — where does this whole area go next, and what should a team actually do differently starting now?

Reading anchor: Mohsenimofidi, Galster, Treude & Baltes (2026), Context Engineering for AI Agents in Open-Source Software*, MSR 2026.*

6 Topic 5: What changes when agents run always-on?

Question to answer: what does an architecture need once the agent no longer waits for a person at a terminal?

Key points. Use three real systems as points on a spectrum — this framing is our own synthesis, not a lineage the sources claim. **Pi** is a session agent: a person starts and supervises each run, and its state is a JSONL session tree. **Pi Durable** (Earendil, 1 October 2026, experimental) is a library, not a new CLI: every model and tool step is a task that checkpoints before moving on, a new process calls `resume()` after a crash, only tools marked safe are rerun, and a `requestId` makes submissions exactly-once. **Hermes** (Nous Research) is always-on: CLI, a long-running chat gateway and ACP clients feed one agent core, a scheduler starts work by itself, memory files are always in context, sessions live in SQLite with full-text search, and compaction at 50% rotates the session. Close with the cost of autonomy (one Hermes user's report: 69% of a 12-

hour session's tokens spent replaying history, issue #5563), the platform turn, and four or five of the harness paper's **18 recommendations**, with the evidence from today's systems.

Hands off to discussion: end with one open question for the room — "which of today's five questions can our own project already answer, and which would we need before letting an agent run unattended?"

Reading anchor: Earendil (2026), Pi Durable*, earendil.com/posts/pi-durable; Nous Research, Hermes Agent documentation; Alejandro AO (2026), Pi and Hermes architecture explainers; Barbaste, Darrigol, Vu & Wiltberger (2026), recommendations section.*

7 The proposed speaking sequence

Time	Slides	Owner and topic
0–2 min	1–2	All: opening and agenda
2–9 min	3–8	Ricardo: Day 1 to Day 2; agent = model + harness (seven subsystems); eleven harnesses from 1 to 109 tools; the agent loop; Pi's minimal core; what eleven harnesses share and lack
9–15 min	9–14	Tomás: the 18-pattern catalogue; eight families; planning, reflection, registry and adapter; decision model
15–21 min	15–19	Simon: why evaluation changes; AgentArcEval process; the Luna accuracy scenario; eleven scenarios and measures; scenarios as a lens on Pi, Pi Durable and Hermes
21–29 min	20–26	Luca: context as a budget; 466 of 10,000 repositories; AGENTS.md content, styles and evolution; what stays in context; subagents
29–37 min	27–33	Matheus: session agent to always-on agent; Pi Durable crash recovery; Hermes architecture; what always-on costs; the platform turn; recommendations; the 90-line harness
37–40 min	34	All: discussion

A note on scope. Each topic is self-contained on purpose: a speaker who only has time to prepare their own section, and read the *Hands off to* notes on either side of it, can still give a coherent talk. Pi, Pi Durable and Hermes recur as evidence, but no slide depends on another speaker's explanation of them.